

Foundations of Convolutional neural network

[Convolutional neural network]

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

1.0 Content Block

LeNet-5, a pioneering 7-level convolutional network by LeCun et al. in 1995, classifies hand-written numbers on checks digitized in 32×32 pixel images. The ability to process higher-resolution images requires larger and more layers of convolutional neural networks, so this technique is constrained by the availability of computing resources. It was superior than other commercial courtesy amount reading systems (as of 1995). The system was integrated in NCR's check reading systems, and fielded in several American banks since June 1996, reading millions of checks per day.

2.0 Content Block

The depth of the output volume controls the number of neurons in a layer that connect to the same region of the input volume. These neurons learn to activate for different features in the input. For example, if the first convolutional layer takes the raw image as input, then different neurons along the depth dimension may activate in the presence of various oriented edges, or blobs of color. Stride controls how depth columns around the width and height are allocated. If the stride is 1, then we move the filters one pixel at a time. This leads to heavily overlapping receptive fields between the columns, and to large output volumes. For any integer $S > 0$, a stride S means that the filter is translated S units at a time per output. In practice, $S \geq 3$ is rare. A greater stride means smaller overlap of receptive fields and smaller spatial dimensions of the output volume. Sometimes, it is convenient to pad the input with zeros (or other values, such as the average of the region) on the border of the input volume. The size of this padding is a third hyperparameter. Padding provides control of the output volume's spatial size. In particular, sometimes it is desirable to exactly preserve the spatial size of the input volume, this is commonly referred to as "same" padding.

3.0 Content Block

Dropout Because networks have so many parameters, they are prone to overfitting. One method to reduce overfitting is dropout, introduced in 2014. At each training stage, individual nodes are either "dropped out" of the net (ignored) with probability $1 - p$ or kept with probability p , so that a reduced network is left; incoming and outgoing edges to a dropped-out node are also removed. Only the reduced network is trained on the data in that stage. The removed nodes are then reinserted into the network with their original weights. In the training stages, p is usually 0.5; for input nodes, it is typically much higher because information is directly lost when input nodes are ignored. At testing time after training has finished, we would ideally like to find a sample average of all possible 2^n dropped-out networks; unfortunately this is unfeasible for large values of n . However, we can find an approximation by using the full network with each node's output weighted by a factor of p , so the expected value of the output of any node is the same as in the training stages. This is the biggest contribution of the dropout method: although it effectively generates 2^n neural nets, and as such allows for model combination, at test time only a single network needs to be tested. By avoiding training all nodes on all training data, dropout decreases overfitting. The method also significantly improves training speed. This makes the model combination practical, even for deep neural networks. The technique seems to reduce node interactions, leading them

to learn more robust features that better generalize to new data.

4.0 Content Block

The spatial size of the output volume is a function of the input volume size W , the kernel field size K of the convolutional layer neurons, the stride S , and the amount of zero padding P on the border. The number of neurons that "fit" in a given volume is then:

5.0 Content Block

Weights Each neuron in a neural network computes an output value by applying a specific function to the input values received from the receptive field in the previous layer. The function that is applied to the input values is determined by a vector of weights and a bias (typically real numbers). Learning consists of iteratively adjusting these biases and weights. The vectors of weights and biases are called filters and represent particular features of the input (e.g., a particular shape). A distinguishing feature of CNNs is that many neurons can share the same filter. This reduces the memory footprint because a single bias and a single vector of weights are used across all receptive fields that share that filter, as opposed to each receptive field having its own bias and vector weighting.

6.0 Content Block

Fully connected layer After several convolutional and max pooling layers, the final classification is done via fully connected layers. Neurons in a fully connected layer have connections to all activations in the previous layer, as seen in regular (non-convolutional) artificial neural networks. Their activations can thus be computed as an affine transformation, with matrix multiplication followed by a bias offset (vector addition of a learned or fixed bias term).

7.0 Content Block

Neural abstraction pyramid The feed-forward architecture of convolutional neural networks was extended in the neural abstraction pyramid by lateral and feedback connections. The resulting recurrent convolutional network allows for the flexible incorporation of contextual information to iteratively resolve local ambiguities. In contrast to previous models, image-like outputs at the highest resolution were generated, e.g., for semantic segmentation, image reconstruction, and object localization tasks.

8.0 Content Block

Fine-tuning For many applications, training data is not very available. Convolutional neural networks usually require a large amount of training data in order to avoid overfitting. A common technique is to train the network on a larger data set from a related domain. Once the network parameters have converged an additional training step is performed using the in-domain data to fine-tune the network weights, this is known as transfer learning. Furthermore, this technique allows convolutional network architectures to successfully be applied to problems with tiny training sets.